CSC 106 - Can Computers Think? W10 - review

CSC 106: Can Computers Think?

Assignments to study

- W2 : for loops, input
- W2 : memory diagrams and tracing
- W3 : math facts
- W3 : conditionals
- W4 : multi-branch conditionals (Reeborg)
- W5 : conditionals
- W5 : while loops
- W6 : ASCII
- W6 : lists and strings
- W6 : Caesar cipher, strings
- W7 : lists, numbers and strings
- W7 : nested lists
- W7 : dictionaries and lists
- W8 : reading/writing files
- W8 : dictionaries for translation
- W9 : author identification
- W9 : recursion, real-life
- W9 : recursion, math

Final Exam Topics

- Values and types
- Statements and expressions
- Operators
- Function calls and definitions
- Loops: conditionals, while and for
- Strings and String operators
- Lists
- Nested Lists
- Files
- Dictionaries
- Recursion
- Sorting
- Search

Values and Types: Discussion

- How many types of values can you think of in Python?
- How would you describe a variable?
- What are are the rules of variable names?
- Which of these variable names are valid? For those that are invalid… why?

```
newVar
a new day
123abc
for
For
email@provider
_once_upon_a_time__________
```

Statements and Expressions: Discussion

- What is an expression?
- What is a statement?
- How would you describe the difference between a statement and and expression?

- What is the correct syntax for the assignment statement?
- How would you describe the assignment statement?
- What is the value of each variable after this code runs?

```
var1 = "apple"
var2 = 2
var3 = "orange-"
var4 = var2 + 10
var2 = 8

print(var1)
print(var2)
print(var3)
print(var4)
```

- Which of the following lines of code are statements?

```
x = 5
x*2
x + 10
a = x - 3
print(x)
```

Operators: Discussion

- What is an operator?
- Think of as many math operators as you can
- What is the order of operations for math operators?
- Which operator would make it easiest to check if a number is even or odd?
- What do these operators do with strings? `+` `*`
- What is the value of each of the following expressions when written in Python?

```
3/2*4
3/(2*4)
(2*3)-4
2**3+1
2**(3+1)
```

```
2**3/4+2
"hello"*2
4//3
5%2
```

```
print(3/2*4)
print(3/(2*4))
print((2*3)-4)
print(2**3+1)
print(2**(3+1))
print(2**3/4+2)
print("hello"*2)
print(4//3)
print(5%2)
```

Function Basics: Discussion

- In your own words, how would you describe a function?
- What is the syntax for calling a function?
- What is the syntax for defining a function?
- Try to come up with the names of at least three built-in functions from Reeborg's World
- Try to come up with the names of at least three built-in functions from Python's Turtle module
- Try to come up with the names of at least three built-in functions from Basic Python (i.e. without Turtle or any other modules or libraries)

Functions with parameters: Discussion

- In your own words, how would you describe the difference between reserved words and built-in functions in Python?
- What is the syntax to call a function that takes parameters?
- What is the syntax to define a function that takes parameters?
- Try to name at least three built-in Python functions that take parameters
- The `int()` function can be used to convert floating point numbers to integers through truncation. What is the difference between rounding and truncation?
  - Can the `int()` function be used to convert a string that represents a floating point number to an integer?
- Does the program below work? Why or why not?

```
def display_product(a, b):
    product = a*b
    print(product)

display_product(5, 10)
```

- Does the program below work? Why or why not?

```
def display_product(a, b):
    product = a*b
    print(product)

display_product(13, 0.5)
```

- Does the program below work? Why or why not?

```
def display_product(a, b):
    product = a*b
    print(product)

display_product(2, "4")
```

- Does the program below work? Why or why not?
  - assume that the user actually enters integer values

```
def display_product(a, b):
    product = a*b
    print(product)
x = 2 #int(input("What's the first factor?"))
y = 4 #int(input("What's the second factor?"))
display_product(x, y)
```

Functions with return values: Discussion

- In your own words, describe the difference between a void function and a fruitful function.
- What is the syntax to store the value returned by a fruitful function?
- What is the syntax to define a function that returns a value?

- What happens if you assign a variable to the result of a void function?
- What will the values of `int_1`, `int_2`, and `int_3` be after the code below runs? Can you explain why?

```
def func1(a, b):
    a = a*b
    return a
int_1 = 2
int_2 = 3
int_3 = func1(int_1, int_2)

print(int_1)
print(int_2)
print(int_3)
```

Conditionals: Discussion

- What is the value of x after this code runs?

```
x = 5
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2

print(x)
```

- What is the value of x after this code runs?

```
x = 1
if x > 2:
    x = -3
    x = 1
else:
    x = 3
    x = 2

print(x)
```

- Given the function definition for is_odd below, what will the result of the function calls following it be?

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False

print(is_odd(1))
print(is_odd(2))
print(is_odd(3.5))
#is_odd("abc")
```

- Consider the following function definition for is_odd. Can you rewrite the function so that it only uses one line after "def is_odd(x):"?

```
def is_odd(x):
    if x % 2 == 1:
        return True
    else:
        return False

print(is_odd(2))
```

```
def is_odd(x):
    return (x % 2) == 1
print(is_odd(2))
```

- What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
if (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

- What gets printed when the following code runs?

```
grade = 100
if (grade >= 90):
    print("You got an A!")
elif (grade >= 80):
    print("You got a B!")
else:
    print("You got another grade.")
```

- How can you fill in the blanks so the following payscale is followed?

	experience
age	0
<18	$15.25
18+	$15.25

```
experience = 0
age = 0

if experience == 0:
    wage = 15.25
elif (_1_):
    if (_2_):
        wage = 17.50
    else:
        wage = 16.25
else:
    wage = 16.0
```

```
experience = 4
age = 17

if experience == 0:
    wage = 15.25
elif (age >= 18 ):
    if (experience > 2):
        wage = 17.50
    else:
        wage = 16.25
else:
    wage = 16.0
```

```
13
14 print(wage)
```

- What will the value of y be after this code runs?

```
1 x = 8
2 y = (x > 4) and ((not(x>8))or(x==5))
3 print(y)
```

While Loops: Discussion

- Under what conditions will the loop keep running? Are there any problems with the loop?

```
1 x = float(input("Enter a number between 10 and 20: "))
2 sum = 0
3
4 while x > 10 or x < 20:
5     sum += x
6     x = float(input("Enter another number between 10 and 20: "))
7
8 print("The sum of all the numbers you entered is", sum)
```

- Under what conditions will the loop keep running? Are there any problems with the loop?

```
1 x = 0
2 sum = 0
3
4 while x > 10 and x < 20:
5     x = float(input("Enter a number between 10 and 20: "))
6     sum += x
7
8 print("The sum of all the numbers you entered is", sum)
```

- Under what conditions will the loop keep running? Are there any problems with the loop?

```
x = float(input("Enter a number less than 0 or bigger than 10: "))
sum = 0

while x > 10 and x < 0:
    x = float(input("Enter a number less than 0 or bigger than 10: "))
    sum += x

print("The sum of all the numbers you entered is", sum)
```

- How would you describe the purpose and function of the following loop?

```
import math

x = float(input("Find the square root of which number? "))

while x < 0:
    x = float(input("Find the square root of which number? "))

print(math.sqrt(x))
```

- Fill in the blanks to make the program function.

```
print("Computing the sum of positive numbers.")

x = float(input("Enter the first number (-1 to stop)"))
sum = 0

while __1___:
    x = float(input("Enter the next number (-1 to stop)"))
    __2___

print(sum)
```

basic string operations

- `+` : concatenation
- `*` : duplication/multiplication
- `in` : checking containment
- `len` : getting length
- `<string name>[<i>]` : indexing
- `<string name>[<start>:<stop>]` : slicing

string operations

- `ord(<char>)` : get the ASCII value of a character
- `chr(<integer>)`: get the character associated with the integer
- `"string".format(<args>)` : format the string
- `"a string".split('<arg>')` : split on ‘’
- `" string  ".strip(<args>)` : remove at the ends
- `"STRing".lower()` : convert all letters to lowercase
- `"STRing".upper()` : convert all letters to uppercase
- `"string".isalpha()` : checks if all characters are letters

Strings: Discussion

- What will the value of mystring be after the following code runs?

```
mystring = ""

for i in range(1, 6):
    mystring += str(i) + " "

print(mystring)
```

- What will the value of mylist be after the following code runs?

```
mystring = "They're celebrating the triumphant return of their heroes."

mylist = mystring.split()
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
mystring = "It is a period of civil war. Rebel spaceships, striking from a
 ↪  hidden base, have won their first victory against the evil Galactic
 ↪  Empire"

mylist = mystring.split(".")
print(mylist)
```

- What will the value of mystring be after the following code runs?

```
mystring = "   abc 123   "

mystring.strip()
print(mystring)
```

- What will the value of mystring be after the following code runs?

```
mystring = "   abc 123   "

mystring = mystring.strip()
print(mystring)
```

- What will the value of mystring be after the following code runs?

```
mystring = "3.14159265359 starts pi, while e starts with 2.7182818284"

mystring = mystring.strip("123456789\. ")
print(mystring)
```

- What will the value of new_string be after the following code runs?

```
mystring = "mdudq fnmmz fhud xnt to, mdudq fnmmz kds xnt cnvm, mdudq fnmmz
 ↪  qtm zqntmc zmc cdrdqs xnt"

new_string = ""
for char in mystring:
    if char.isalpha():
        new_string += chr(ord(char) + 1)
    else:
        new_string += char
print(new_string)
```

list operations

- `+` : concatenation
- `*` : duplication/multiplication
- `in` : checking containment
- `len` : getting length
- `[]` : indexing

- `append()` : appending
- `extend()` : extending
- `<list>[start:stop]` : slicing

Lists: Discussion

- What will the value of mylist be after this code snippet has been executed?

```
mylist = ["a string", 123, False, 4.5]
mylist[2] = True

print(mylist)
```

- What will the code below print to the screen?

```
numlist = [1, 231, 85, 4]
for num in numlist:
    print(num, end= "")
```

- What will the value of nums2 be after this code snippet runs?

```
nums1 = [1, 3, 5, 7, 9, 11, 13]
nums2 = nums1[2:5]
print(nums2)
```

- What will the value of nums3 be after this code snippet runs?

```
nums1 = [1, 3, 5]
nums2 = [2, 8, 10]
nums3 = nums1 + nums2
print(nums3)
```

- What will the code below print to the screen?

```
mylist = [1] * 4
print(mylist)
```

- What will the value of nums be after this code snippet runs?

```
nums = [1, 3, 5]
nums.append(8)
print(nums)
```

- What will the value of nums be after this code snippet runs?

```
nums = [1, 3, 5]
nums.append([8, 9])
print(nums)
```

- What will the value of nums be after this code snippet runs?

```
nums = [1, 3, 5]
nums.extend([8, 9])
print(nums)
```

- Will the following code run? If not, why?

```
mylist = [7, 8, 9, "123", "run"]
mylist.sort()
print(mylist)
```

- Write a snippet of code to compute the average of the numbers contained in the list nums.

```
nums = [991, 563, 647, 446, 929]
```

- What will the value of names be after this code runs?

```
def sort_and_pop(list_in):
     list_in.sort()
     list_in.pop()

names= ["Zeke", "eren", "Sasha", "mikasa"]
sort_and_pop(names)
print(names)
```

- What will the following code print?

```
def list_change(in_list):
    in_list = [9, 10, 11]
    print(in_list)

mylist = ["W", "A", "H"]
print(mylist)
list_change(mylist)
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    print(in_list)
    in_list = in_list + [4, 5, 6]

mylist = ["e", "f", "g"]
list_change(mylist)
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    print(in_list)
    in_list.extend(["a", "b", "c"])

mylist = [1, 2, 3]
list_change(mylist)
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    in_list.append("e")
    in_list.append("f")
    in_list.append("g")
    print(in_list)

mylist = [4, 5, 6]
list_change(mylist)
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
def list_change(in_list, first_char):
    start = ord(first_char)
    for i in range(len(in_list)):
        in_list[i] = chr(start + i)*(i + 1)

mylist = [1, 2, 3]
list_change(mylist, "b")
print(mylist)
```

- What will the value of mylist be after the following code runs?

```
def list_change(in_list):
    in_list = in_list[1:4]

mylist = ["a", "b", "c", "d", "e", "f", "g"]
list_change(mylist)
print(mylist)
```

nested lists: discussion

Write a loop or set of nested loops to print the following nested list in a grid.

```
mylist = [[1, 3, 5],
[2, 4, 6, 8, 10],
[3, 5, 7, 11],
[2, 4, 8, 16, 32, 64]]
```

- What will the value of mylist be after the following code runs?

```
mylist = []

for i in range(5):
    mylist.append([])
    for j in range(3):
        mylist[i].append((i+1)**j)

print(mylist)
```

- What will the following code print?

```
mylist = [[1, 3, 5],
[2, 4, 6, 8, 10],
[3, 5, 7, 11],
[2, 4, 8, 16, 32, 64]]

for line in mylist:
    if 3 in line:
        print(line)
```

- What will the following code print?

```
mylist = [["a", "b", "c"],
          ["d", "e", "f"],
          ["abc", "def", "ghi"],
          ["ball", "a", "hall"],
          "abcdefg"]

for line in mylist:
    if 'a' in line:
        print(line)
```

- What will the value of mylist be after the following code runs?

```
list1 = []
list2 = []
mylist = []

for num_1 in list1:
    line = []
    for num_2 in list2:
        line.append(num_1*num_2)
    mylist.append(line)

print(mylist)
```

- What will the following code print?

```
mylist = [["night", "fear", "love", "here", "flier"],
          ["gift", "gong", "hand", "bound", "playa"],
          ["greetings", "fire", "wyvern", "tried"],
          ["yacht", "room", "dour"],
```

```
          ["unified", "opinions"]]

for i in mylist:
    for j, word in enumerate(i):
        print(word[j], end="")
    print(end=" ")
```

- Are there any problem with the following code? If so, explain.

```
mylist = []


for i in range(1, 11):
    list1 = []
    for j in range(1, 11):
        list2 = []
        for k in range(1, 11):
            list2.append(i*j*k)
        list1.append(list2)
    mylist.append(list1)

print(mylist)
```

- There is a problem with the behavior of this code. Please explain the problem.

```
def record_sales(in_list):
    sale = float(input("Enter the next sale (-1 to stop): "))
    while not (sale < 0):
        sale = float(input("Enter the next sale (-1 to stop): "))
        in_list.append(sale)

sales_records = []
enter_record = input("Enter records for a new day (y/n)? ")
while enter_record.lower()[0] == "y":
    sales_records.append([])
    record_sales(sales_records[len(sales_records)-1])
    enter_record = input("Enter records for a new day (y/n)? ")
```

- Fill in the blanks, so this asks for new entries until “n” is entered

```
def add_record():
    fname = input("First name: ")
    lname = input("Last name: ")
    phone = input("Phone number: ")
    return [fname, lname, phone]

customer_list = []
enter_record = input("Enter new record (y/n)? ")
while enter_record.lower()[0] != "n":
    customer_list.append(.............)
.....................................................

print(customer_list)
```

Basic File Operations

- Getting access
  - `open("<filename>",mode="a/r/w")` : get access to the file
- Reading contents
  - `read()` : return the remaining contents as a string
  - `readline()` : return the next line's contents as a string
  - `readlines()` : return the remaining contents as a list of strings
- Writing new stuff
  - `write(<a string>)` : write a single string to the file
  - `print(<args>,file=<f>)` : write information to the file with print formatting
- Closing it: the most important one… do not forget
  - `close()` : close the file and save its contents to long-term storage

Files: Discussion

test_file.txt

```
Three Rings for the Elven-kings under the sky,
Seven for the Dwarf-lords in their halls of stone,
```

```
Nine for Mortal Men doomed to die,
One for the Dark Lord on his dark throne
In the Land of Mordor where the Shadows lie.
One Ring to rule them all, One Ring to find them,
One Ring to bring them all, and in the darkness bind them,
In the Land of Mordor where the Shadows lie.
```

- What are the three modes which can be used when opening a file?
- What type of data does contents hold after this code runs?

```
1 infile = open("test_file.txt", "r")
2 contents = infile.readlines()
3 infile.close()
```

- What type of data does contents hold after this code runs?

“‘{python{ infile = open(“test_file.txt”, “r”) contents = infile.readline() infile.close()

print(type(contents)) print(contents)

```
* What type of data does contents hold after this code runs?

::: {.cell execution_count=57}
``` {.python .cell-code}
infile = open("test_file.txt", "r")
contents = infile.read()
infile.close()

print(type(contents))
print(contents)

:::
```

- test_file.txt

```
Three Rings for the Elven-kings under the sky,
Seven for the Dwarf-lords in their halls of stone,
Nine for Mortal Men doomed to die,
One for the Dark Lord on his dark throne
In the Land of Mordor where the Shadows lie.
```
```

```
One Ring to rule them all, One Ring to find them,
One Ring to bring them all, and in the darkness bind them,
In the Land of Mordor where the Shadows lie.
```

- What will the following code display?

```
infile = open("test_file.txt", "r")
lines = infile.readlines()
infile.close()

for line in lines:
    print(line.strip())
```

- How many strings will be contained in lines after this code runs?

```
infile = open("test_file.txt", "r")
lines = infile.readlines()
infile.close()

for line in lines:
    print(line.strip())
```

file2.txt

```
This was a triumph.
I'm making a note here:
huge success.
It's hard to overstate
My satisfaction.
Aperture Science.
We do what we must
Because we can.
For the good of all of us.
Except the ones who are dead.
```

- What will the following code print?

```
infile = open("file2.txt", "r")
contents = infile.read()
infile.close()
contents = contents.split("\.")

for item in contents:
    print(item.strip())
```

- What will the file outfile contain after this code runs?

```
outfile = open("out.txt", "w")

for i in range(4):
    print(i, i**2, file=outfile)
outfile.close()
```

Basic Disctionary Operations

- Consider the following dictionary: `d = {'z': 430, 116: 'e'}`
- `d.keys()` : get all of the dictionary's keys
- `d.values()` : get all of the dictionary's values
- `d.items()` : get all of the dictionary's key:value pairs

dictionary: discussion

- What will the following code print?

```
d = {'briar':128, 'logan':45, 'skylar':77, 'jadyn':144}

print(d["logan"])
```

- What of the following is best implemented as a dictionary instead of a list?

1. The amount of time elapsed between presses of a button
2. The names on a class roster
3. The names of trees and descriptions of their leaves
4. 42 randomly generated floating point numbers

- What will the following code print?

```
d = {'bamboo':" ", 'mirror':" ", 'day':" ", 'book':" "}

print(d["bamboo"])
```

- Will the following code run? Why or why not?

```
d = {}

usr_in = input("Enter a key:value pair")
usr_in = usr_in.split(";")
while len(usr_in) == 2:
    d[usr_in[0]] = usr_in[1]
    usr_in = input("Enter a key:value pair")
    usr_in = usr_in.split(";")

print(d)
```

- Assume that d was defined as a dictionary previously. How would you describe what this code does?

```
d = {}
usr_in = input("Enter the word you want to look up")
while len(usr_in) >0:
    if usr_in in d.keys():
        print(d[usr_in])
    else:
        print("word unknown")
        d[usr_in] = input("Please provide a definition")
        usr_in = input("Enter the word you want to look up")

print(d)
```

- Given the snippet of code below, write another snippet of code to add the key "pine" to the dictionary with the value 45.

```
trees = {'oak': 3, 'walnut': 97, 'maple': 24, 'birch': 1}
```

- What does the following code print?

```
names = {'Janice': 5, 'Emily': 3, 'John': 7, 'Eleanor': 2}
list_o_names = []
for name in names:
    if names[name] > 5:
        list_o_names.append(name)
print(list_o_names)
```

- This works:
  - `d[("a", 123, False)] = True`
- But this does not:
  - `d[("a", [1,2,3], False)] = True`
- Why?
- Assume that d was defined as a dictionary previously. How do you think this dictionary is intended to be used?

```
d = {}
in_str = "George Lucas: Star Wars- Episode IV - A New Hope"
in_str = in_str.split(":")

d[in_str[0]].append(in_str[1])
```

- Is the following code valid? Why or why not? How do you think this dictionary is intended to be used?

```
d = {}
d["lotr"] = {"l1" : "Three Rings for the Elven-kings under the sky",
     "l2" : "Seven for the Dwarf-lords in their halls of stone",
     "l3" : "Nine for Mortal Men doomed to die",
     "l4" : "One for the Dark Lord on his dark throne"}

print(d)
```

- Is the following code valid? Why or why not?
  - What do you think this is being used for?

```
d = {}
d["    "] = {"b1": "       ",
      "b2" : "        ",
      "b3" : "       ",
      "b4" : "        "}
print(d)
```

- Is the following code valid? Why or why not?
  - What do you think this is being used for?

```
d = {}
d["img1"] = "https://i.imgur.com/cC2RzZk.jpg"
d["img2"] = "https://i.imgur.com/AIIze1P.jpg"
d["img3"] = "https://i.imgur.com/SJActxD.jpg"
d["img4"] = "https://i.imgur.com/a2dXlxd.jpg"

print(d)
```

- Is the following code valid? Why or why not?

```
d = {}
d[0] = 123
d[1] = "abc"
d[2] = "xyz"
d[3] = False

print(d)
```

- Is the following code valid? Why or why not?
  - Assume both d and e were previously defined as dictionaries.

```
e={}
for key in d.keys():
    e[key] = d[key]*2

print(d)
print(e)
```

Which of the following is a difference between lists and dictionaries?

1. List elements cannot be mutable, but dictionary values can be.
2. Assigning to an index that does not exist in a list is an error, but assigning a value to a key that doesn't exist in a dictionary is not
3. A list can contain a dictionary as one of its elements, but a dictionary cannot contain a list as one of its value
4. There is a dict constructor that creates a dictionary from a suitable object, but there is no list constructor that does the same.

Recursion: Discussion

- In your own words, describe recursion as it pertains to computer programming.
- Can you think of any examples in real life where you see recursive structures?
- In your own words describe halting as it pertains to computer programming.
- Will the following recursive function always halt? Why or why not?

```
def func(num, adjust=0):
    if num % 13 ==0:
        return adjust
    else:
        return func(num+1, adjust+1)
```

- Will the following recursive function always halt? Why or why not?

```
def func(num):
    if num <= 1:
        return num
    else:
        return num + func(num+1)
```

- Below is an example of a recursive function to calculate Fibonacci numbers.
  - What do you think the pros and cons of this method are?

```
def fib(num):
    if num <= 1:
        return num
    else:
        return fib(num-1) + fib(num-2)
```

- Below is another example of a recursive function to calculate Fibonacci numbers.
  - Is this method better or worse than the last one? Why?

```
def fib(num, val=1, prev=0):
    if num == 0:
        return num
    return fib(num-1, val+prev, val)
```

Remember at least one of the sort methods we discussed

- Bubble Sort
- Merge Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Bucket / Bin Sort
- Linear Search
- Binary Search

Final Exam Topics

- Values and types
- Statements and expressions
- Operators
- Function calls and definitions
- Loops: conditionals, while and for
- Strings and String operators
- Lists
- Nested Lists
- Files
- Dictionaries
- Recursion
- Sorting
- Search